Deliverable 2 - INF201

Submission deadline: 30th Oct 2026

Hand-in information:

- This is a group submission, so one person submits on behalf of the pair/group
- Submit the Jupyter notebook (ipynb) *and* a pdf of the same notebooks
  - Should contain code, results/plots and discussion in markdown cells
- Also submit the output files generated by your code
- Include a link to your GitHub repository and a log of the commits
  - To generate the log, paste this into the terminal after your last commit: `git log --graph --pretty=format:"%h - %an, %ar : %s" > git_log.txt`
  - The repository should contain a `requirements.txt` with information on your virtual environment
  - Note: Remember to create a new virtual environment for each deliverable
- Include a discussion of GenAI use, for example a summary of the chat log as you have done earlier
  - This can be a separate file or be included in the Jupyter notebook
- Make a short (5 mins) video discussing your code and results
  - Note that you both have to contribute to the video
  - Either by recording it together
  - Or recording parts separately and editing it together

Scenario - Campus Coffee Shop Sales Pipeline

You have been hired as a Python developer for the Campus Coffee Shop Chain. The chain operates several coffee stands across different campus buildings. This is an international campus, so the stands accept payment in several currencies.

Each cash register saves transaction logs as text files. However, the legacy system has several issues:

- Mixed Character Encodings: Older registers write text files using ISO-8859-1 (Latin-1) encoding, while newer registers write files using UTF-8.
- System Noise: Registers intersperse valid sales lines with printer and network debug log lines.
- Nested Folder Hierarchy: Log files are stored across nested subdirectories organized by year, store, and register ID.

- Data Errors: System glitches record occasional extreme price outliers (e.g., $999.00), and prices in NOK are by error registered using EUR-scale values (e.g., 4.50 NOK instead of 45.00 NOK).

Your task is to set up Python code that discovers all log files, cleans the transaction data, calculates price metrics and generates summary reports.
Note: In this assignment, you write the functions you will use in tasks 1-4, and then put everything together in task 5. You should therefore read through the entire assignment text before you start, to fully understand what you will use each function for.

Overview of inputs and outputs for this assignment

Inputs (in `data/` folder)

- You generate the input data on you own laptop by running the script `generate_data_for_d2.py`
- `data/store_config.json` : A nested JSON file containing store metadata.
- `data/2026/...` : A folder hierarchy containing subdirectories with .log files in mixed character encodings.

Outputs (files your code should generate)

- `cleaned_transactions.csv` : A CSV table containing all parsed, outlier-cleaned, and currency-corrected transactions.
- `hourly_revenue.json` : A JSON report containing total revenue per hour per item.
- `status_pivot_table.csv` : A CSV file containing a pivot table of total transaction revenue grouped by item across status categories.
- `item_refund_ratios.csv` : A CSV table containing refund ratios for each item type.

Task 1: File ingestion and regular expressions

1. Write a robust file reader function:
   - Use `pathlib.Path` to handle file paths.
   - The function should be able to handle both `UTF-8` and `ISO-8859-1/Latin-1` encoding. You may assume that these are the only two encodings the files can have. Note that the function should handle the encoding, you may not hard-code which encoding each file has.
   - Return a tuple containing two strings: `(file_text_content, encoding_used)` .
2. Write a log parser function that uses regular expressions. Log files contain valid sales lines mixed with system noise.
   Valid line format:
   `[2026-10-15 08:30:00] [POS_01] item=Latte price=4.50 EUR (SUCCESS)`
   Noise lines to ignore:

`--- PRINTER PAPER LOW WARNING ---`

`=== NETWORK SYNC OK ===`

- Construct a regular expression (`re` module) to capture the 6 key fields from valid lines:
  - `timestamp` (string, format: `YYYY-MM-DD HH:MM:SS`)
  - `register_id` (string, e.g., `POS_01`)
  - `item` (string, e.g., `Latte`)
  - `price` (float, e.g., `4.50`)
  - `currency` (string, e.g., `EUR` , `NOK` , `USD`)
  - `status` (string: `SUCCESS` , `REFUND` , or `CANCELLED`)
- Return a list of Python dictionaries, where each dictionary represents one valid transaction record.

In [5]:
```
import re
import json
from pathlib import Path
import numpy as np
import pandas as pd
```

In [6]:
```
#Task 1.1
def read_log_file(path) -> tuple[str, str]:
    #read a text file, trying UTF-8 first and falling back to latin-1
    path = Path(path)
    try:
        text = path.read_text(encoding="utf-8")
        return text, "utf-8"

    except UnicodeDecoderError:
        text = path.read_text(encoding="iso-8859-1")
        return text, "iso-8859-1"
```

In [7]:
```
text, enc= read_log_file("data/2026/library_cafe/POS_01/register_sales.log")
print("Encoding:", enc)
print(text[:400])
```

```
Encoding: utf-8
[DEBUG] BARCODE SCANNER REBOOTED
[2026-10-15 07:01:52] [POS_01] item=Sandwich price=7.23 USD (SUCCESS)
[2026-10-15 07:03:37] [POS_01] item=Sandwich price=7.98 USD (SUCCESS)
[2026-10-15 07:04:35] [POS_01] item=Espresso price=3.28 EUR (SUCCESS)
[2026-10-15 07:06:48] [POS_01] item=Latte price=4.59 EUR (REFUND)
[2026-10-15 07:08:27] [POS_01] item=Latte price=4.90 USD (SUCCESS)
[DEBUG] BARCODE SCANNER
```

In [8]:
```
LINE_PATTERN = re.compile( #remompile
    r"^\[(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\]\s+" # Tid [2026-10-15
    r"\[(?P<register_id>POS_\d+)\]\s+" #Namn på kassan
    r"item=(?P<item>\w+)\s+" #Namn på ting
    r"price=(?P<price>\d+\.\d+)\s+" #Pris
    r"(?P<currency>[A-Z]{3})\s+" #Currency
    r"\((?P<status>SUCCESS|REFUND|CANCELLED)\)$" #Vilken status har: succuss, refun

)
def parse_log(text: str) ->list[dict]: #Function får en input som str
    #Extract valid transection lines from log text; ignore noise lines
    records =[] # En list av alla transactioner
    for line in text.splitlines():
        match = LINE_PATTERN.match(line.strip()) #delete alla onödiga karaktärer
        if match:
            record = match.groupdict() #Ge tillbake alla grupper som har namn
            record["price"] = float(record["price"])
            records.append(record)
    return records
```

In [9]:
```
records = parse_log(text)
print("Number of valid records:", len(records))
print("Lines in file:", len(text.splitlines()))
print(records[0:6])
```

```
Number of valid records: 80
Lines in file: 90
[{'timestamp': '2026-10-15 07:01:52', 'register_id': 'POS_01', 'item': 'Sandwich', '
price': 7.23, 'currency': 'USD', 'status': 'SUCCESS'}, {'timestamp': '2026-10-15 07:
03:37', 'register_id': 'POS_01', 'item': 'Sandwich', 'price': 7.98, 'currency': 'USD
', 'status': 'SUCCESS'}, {'timestamp': '2026-10-15 07:04:35', 'register_id': 'POS_01
', 'item': 'Espresso', 'price': 3.28, 'currency': 'EUR', 'status': 'SUCCESS'}, {'tim
estamp': '2026-10-15 07:06:48', 'register_id': 'POS_01', 'item': 'Latte', 'price': 4
.59, 'currency': 'EUR', 'status': 'REFUND'}, {'timestamp': '2026-10-15 07:08:27', 'r
egister_id': 'POS_01', 'item': 'Latte', 'price': 4.9, 'currency': 'USD', 'status':
'SUCCESS'}, {'timestamp': '2026-10-15 07:10:45', 'register_id': 'POS_01', 'item': 'C
roissant', 'price': 3.93, 'currency': 'EUR', 'status': 'SUCCESS'}]
```

Discussion - Task 1 file reader: Funktionen försöker först avkoda filen som UTF-8. Om det ger ett UnicodeDecodeError faller den tillbake på ISO-8859-1 (Latin-1). Ordningen är viktig: Latin kan avkoda alla bytesekvenser utan fel, så om vi provde latin-1 först skulle det alltid lyckas även för UTF-8 filer, och specialtecken skulle läsa fel. I vår körning lästes alla 9 filer som UTF-8, trots att generatorn sparade några av dem som ISO-8859-1, det är väntat: loggfilerna innehåller bara vanliga asci tecken, och ascii bytes är identiska i båda kodningana. Reservlösning skulle bara användas om en latin-1-fil innehöll icke-ascii tecken. Regex-pasern: Vi använda namngivna grupper (?P...) så att match.groupdict() direkt ger en ordbok med de sex fälten. Ankarna ^ och $ kräver att hela raden matcher mönstret, så brusrader som '--- printer paper low ---' och '[debug] barcode scanner rebooted ' hoppas över automatisk. Priset omvandlas till float efter matchningen. Varje fil gav exakt 80 giltiga poster, vilket stämmer med generatorn (80 föreläsningsrader per fil)

Task 2: Recursive utility functions

1. Write a recursive directory search function that finds all log files in a nested directory and returns a list of `pathlib.Path` objects:
   - Convert the input path to a `pathlib.Path` object.
   - Iterate through items using `Path.iterdir()`.
   - If an item is a directory, recursively search inside it.
   - If an item is a file with a matching extension (`.log`), add its Path object to a list.
   - Constraint: Do NOT use `Path.glob()`, `Path.rglob()`, or `os.walk()`. You must implement explicit recursion.
2. Metadata about the different stores can be found in the `data/store_config.json` file. This file contains deeply nested dictionaries and lists (see example below). Write a recursive function that:
   - Traverses any combination of nested dictionaries and lists found in this file.
   - Returns a set (type hint: `set[str]`) of all register id strings where `"active": true`.

In [10]:
```python
#Task 2.1
def find_log_files(directory) -> list[Path]:
    # Recursively find all .log files in a directory and its subdirectories
    directory = Path(directory) # Det kasnke vara en str som data då ändrar vi
    found = []
    for item in directory.iterdir():
        if item.is_dir():
            found.extend(find_log_files(item))
        elif item.is_file() and item.suffix == ".log":
            found.append(item)
    return found
```

In [11]:
```python
log_files = find_log_files("data")
print("Number of log files:", len(log_files))
for f in log_files:
    print(f)
```

```
Number of log files: 9
data\2026\library_cafe\POS_01\register_sales.log
data\2026\library_cafe\POS_02\register_sales.log
data\2026\library_cafe\POS_03\register_sales.log
data\2026\science_building\POS_01\register_sales.log
data\2026\science_building\POS_02\register_sales.log
data\2026\science_building\POS_03\register_sales.log
data\2026\student_union\POS_01\register_sales.log
data\2026\student_union\POS_02\register_sales.log
data\2026\student_union\POS_03\register_sales.log
```

In [12]:
```python
#Task 2.2
def find_active_registers(data) -> set[str]:
    # Recursively collect ids of all registers with "active": true.
    active =set()
    if isinstance(data, dict):
```

```
        if "id" in data and data.get("active") is True:
            active.add(data["id"])
        for value in data.values():
            active |= find_active_registers(value)
    elif isinstance(data, list):
        for element in data:
            active |= find_active_registers(element)
    return active
```

In [13]:
```
config_path = Path("data") / "store_config.json"
config = json.loads(config_path.read_text(encoding="utf-8"))

active_registers = find_active_registers(config)
print("Active registers:", active_registers)
```

```
Active registers: {'POS_03', 'POS_01', 'POS_02'}
```

Diskussion 2: Rekrusiv filsökning: Functionen går igenom varje objekt i en mapp med Path.iterdir(). Om objektet är en mapp anropar funktionen sig själv på den mappen; Om det är en .log-fil läggs den till i listan. Rekursionen stannar när en mapp inte har några under mappar(basfall). Vi använde extend() i stället för append() så att resultatet blir en platt lista i stället för en nästlad. Funktionen hittade alla 9 loggfiler och ignorerade store-config.json. Aktiva kassor: Konfigurationsfilen hanteras med samma ide, men med tre fall: om datan är en ordbok kontrollerar vi om den är en aktiv kassa och försätter sedan rekrusivt in i alla dess värden; om den är en lista går vi rekrusivt in i varje element; allt annat(strängar, tal, booleska värden) ger en tom mängd. Det betyder att funktionen fungerar för vilket nästlingsdjup som helst, inte bara för strukturen i vår fil. Resultatet blev {POS_01, POS_02, POS_03}; POS_04 uteslöts eftesom "activa": false. Observatorn: Kass-ID:na POS_01-pos_03 återanvänds i alla tre butiker, och konfigurationen beskriver bara två av de tre butikerna. Filtreringen på activa kassor tog därför i praktiken inte bort några transaktioner. I ett verktligt system borde kassorna ha unika ID:n (till exempel med butikens namn inkluderat).

Example config.json:

```
{
  "company": "Campus Coffee",
  "regions": [
    {
      "region_name": "Main Campus",
      "stores": [
        {
          "name": "Library Cafe",
          "registers": [
            {"id": "POS_01", "active": true},
            {"id": "POS_02", "active": false}
          ]
        }
      ]
    }
  ]
}
```

Task 3: Functions for cleaning and analysing data using NumPy

1. Write a function for removing price outliers that:
   - Converts input price data to a 1D NumPy float array.
   - Detects hardware glitch outliers (999.00) where a value deviates from the mean by more than 3 standard deviations:

$$|x - \mu| > 3\sigma$$

   - Replaces detected outlier values with the median of the valid prices.
   - Returns the cleaned 1D array.
2. Write a function `compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarray) -> np.ndarray` that:
   - Computes the element-wise refund ratio using NumPy array operations:

$$\text{Refund Ratio} = \frac{\text{Refund Count}}{\text{Sales Count} + 10^{-6}}$$

   - The addition of $\epsilon = 10^{-6}$ prevents zero-division errors.

In [14]:
```
#Task 3.1
def remove_outliers(prices) -> np.ndarray:
    #Replace value more than 3 standard deviations from the mean with the median of
    x = np.asarray(prices, dtype=float).ravel()
    mean = x.mean()
    std = x.std()
    is_outlier = np.abs(x - mean) > 3 * std
    median_valid = np.median(x[~is_outlier])
```

```
    return np.where(is_outlier, median_valid, x)
```

In [15]:
```
#Task 3.2
def compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarray) -> np.n
    # Element-wise refund ratio: epsilon avoids division by zero
    sales = np.asarray(sales_count, dtype = float)
    refunds = np.asarray(refund_count, dtype = float)
    return refunds / (sales + 1e-6)
```

In [16]:
```
# Test
test_prices = [4.2, 3.1, 7.4, 3.8, 4.5, 3.0]
print("Before:", test_prices[-3:])
print("After:", remove_outliers(test_prices)[-3:])

sales = np.array([10, 20, 0])
refunds = np.array([2, 5, 1])
print("Refund ratios:", compute_refund_ratio(sales, refunds))
```

```
Before: [3.8, 4.5, 3.0]
After: [3.8 4.5 3. ]
Refund ratios: [1.99999980e-01 2.49999988e-01 1.00000000e+06]
```

Diskussion 3 Bortaggning av extremvärden: Ett pris markeras som som extremvärde om $|x - \mu| > 3\sigma$. Vi skapar en boolesk mask, beräknar medianen av endast de giltiga priserna och använder np.where för att alla ersätta extremvärderna med denna median. Alla operationer är vektoriserade, så ingen python-loop behövs. FelVärderna 999.00 uptäcks eftersom de ligger mycket långt från resten av datan: de drar upp medelvärdet till ungefär 30 och standard avvikelsen till ungefär 170, men 999 ligger ändå mycket mer än från medelvärdet. Begränsning vi observerade: När vi testade med en mycket kort lista([4.2, 3.1, 7.4, 999.0]) upptäcktes 999 inte. Med så få värden blåser extremvärdet själv upp σ så mykcet att det hamnar innanför 3σ-gränsen. 3σ-regeln fungerar därför bäst när det finns mycket data och bara få extremvärden. Returkvot: Funktionen dividerar elemntvis och lägger till $\varepsilon = 10^{-6}$ i nämnaren. Vårt test visade att ett försäljningsantal på noll ger ett mycket stort tal ($\approx 10^{6}$) i stället för ett `ZeroDivisionError` .

Task 4: Data analysis using Pandas

1. Write a function that:
   - Converts the list of transaction dictionaries read from the log files into a `pandas.DataFrame` .
   - Converts the `timestamp` column to `datetime` and sets it as the DataFrame index.
   - Sorts the DataFrame chronologically by timestamp.
2. Write a function that has the DataFrame you just created as input, groups data by item and resamples transactions in 1-hour windows (1h), calculating total price (sum).
3. Write a function that has the DataFrame as input and creates a pivot table displaying total transaction revenue grouped by item across status categories (`SUCCESS` ,

REFUND , CANCELLED).

4. Write a function that has the DataFrame as input and calculates the refund ratio per item by calling the function you made in task 3.

In [17]:
```
all_records = []
for f in find_log_files("data"):
    text, enc = read_log_file(f)
    all_records.extend(parse_log(text))

print("Total records:", len(all_records))
```

```
Total records: 720
```

In [18]:
```
#Task 4.1
def build_dataframe(records: list[dict]) -> pd.DataFrame:
    #Convert transaction dicts to a DataFrame indexted and sorted by timestamp
    df = pd.DataFrame(records)
    df["timestamp"] = pd.to_datetime(df["timestamp"])
    df = df.set_index("timestamp")
    df = df.sort_index()
    return df
```

In [19]:
```
#   Task 4.2
def hourly_revenue(df: pd.DataFrame) -> pd.DataFrame:
    # Total price per item in 1-hour windows (rows = hours, columns = items)
    hourly = df.groupby("item")["price"].resample("1h").sum()
    return hourly.unstack(level="item").fillna(0)
```

In [20]:
```
# Task 4.3
def status_pivot(df:pd.DataFrame) -> pd.DataFrame:
    #Total revenue per item (rows) for each statue(columns)
    return pd.pivot_table(
        df,
        values = "price",
        index = "item",
        columns = "status",
        aggfunc = "sum",
        fill_value = 0,
    )
```

In [21]:
```
#Task 4.4
def item_refund_ratios(df:pd.DataFrame) -> pd.DataFrame:
    #refund ratio per item, using compute_refund_ratio from task 3.
    counts = pd.crosstab(df["item"], df["status"])
    counts = counts.reindex(columns=["SUCCESS", "REFUND", "CANCELLED"], fill_value=

    ratio = compute_refund_ratio(
        counts["SUCCESS"].to_numpy(),
        counts["REFUND"].to_numpy(),

    )
    return pd.DataFrame({
        "sales_count": counts["SUCCESS"],
        "refund_count": counts["REFUND"],
```

```
        "refund_ratio": ratio,
    })
```

In [22]:
```
# Test
df = build_dataframe(all_records)
print(df.head())
print()

hourly = hourly_revenue(df)
print(hourly)
print()

pivot= status_pivot(df)
print(pivot)
print()

ratios = item_refund_ratios(df)
print(ratios)
```

```
                    register_id        item  price currency     status
timestamp
2026-10-15 07:00:21      POS_01      Muffin   3.20      NOK    SUCCESS
2026-10-15 07:00:29      POS_03  Cappuccino   3.93      NOK    SUCCESS
2026-10-15 07:01:05      POS_02  Cappuccino   4.19      USD    SUCCESS
2026-10-15 07:01:18      POS_02   Croissant   4.17      NOK    SUCCESS
2026-10-15 07:01:36      POS_03      Muffin   3.68      EUR  CANCELLED

item                 Cappuccino  Croissant  Espresso    Latte   Muffin  \
timestamp
2026-10-15 07:00:00     2217.56    2200.33   1174.14  2251.04  2201.14
2026-10-15 08:00:00     4232.45     204.38   1159.60  3186.09   204.28
2026-10-15 09:00:00       32.99      23.19     21.93  1030.09  1030.16

item                 Sandwich
timestamp
2026-10-15 07:00:00   1387.32
2026-10-15 08:00:00   3430.86
2026-10-15 09:00:00   1066.75

status      CANCELLED   REFUND  SUCCESS
item
Cappuccino    2083.30  1089.70  3310.00
Croissant     1071.90    89.45  1266.55
Espresso      1056.19  1061.58   237.90
Latte         2071.98  2097.53  2297.71
Muffin         106.31    76.24  3253.03
Sandwich      2147.56  2148.99  1588.38

            sales_count  refund_count  refund_ratio
item
Cappuccino           78            22      0.282051
Croissant            72            23      0.319444
Espresso             79            21      0.265823
Latte                68            24      0.352941
Muffin               75            22      0.293333
Sandwich             80            22      0.275000
```

Diskussion – Uppgift 4

Kolumnen timestamp omvandlas till `datetime` och används som index. Sorteringen behövs eftersom filerna läses en kassa i taget, så de råa posterna är inte i kronologisk ordning. Ett datetime-index krävs också för `resample("1h")`.

- **Intäkt per timme:** `groupby("item") + resample("1h") + sum()`, sedan `unstack()` så att varje artikel blir en kolumn. Timmar utan försäljning för en artikel fylls med 0.
- **Pivottabell:** totalt pris per artikel (rader) och status (kolumner).
- **Returkvot:** `pd.crosstab` räknar transaktioner per artikel och status. Vi anropar `compute_refund_ratio` från uppgift 3 med dessa antal.

Antagande: Uppgiften definierar inte "sales count". Vi valde antalet `SUCCESS`-transaktioner, så kvoten betyder "returer per lyckad försäljning". Att använda alla transaktioner som nämnare vore också ett rimligt val och skulle ge lägre kvoter.

Task 5: Create full workflow

Combine the functions you have made into a workflow that does the following:

1. Recursively find all log files and extract active register IDs.
2. Ingest text logs and parse records into a list of dictionaries.
3. Build the initial Pandas DataFrame.
4. Apply NumPy outlier cleaning to remove 999.00 price glitches.
5. Apply currency correction: multiply price by 10.0 for all transactions where `currency == "NOK"`.
6. Generate resampled reports, pivot tables and refund ratios.
7. Save all output files listed in the introduction (Scenario: Overview of inputs and outputs for this assignment).

In [23]:
```
def run_pipeline(data_dir="data", out_dir="output"):
    #Full workflow: find logs, parse, clean, analyse and save all report
    data_dir = Path(data_dir)
    out_dir = Path(out_dir)
    out_dir.mkdir(exist_ok = True)

    #1) Find log and active registers
    log_files = find_log_files(data_dir)
    config = json.loads((data_dir / "store_config.json").read_text(encoding="utf-8"
    active = find_active_registers(config)
    print(f"Found {len(log_files)} log files. Active registers: {sorted(active)}")

    #2) read and parse all logs
    records = []
    for f in log_files:
        text, enc = read_log_file(f)
        file_records = parse_log(text)
```

```
        records.extend(file_records)
        print(f"{f} [{enc}] -> {len(file_records)} records")

    #3) Build DataFrame, keep only avtive registers
    df = build_dataframe(records)
    df = df[df["register_id"].isin(active)].copy()

    #4) remove price outliers (999.00 glitches)
    n_outliers = (df["price"] == 999.0).sum()
    df["price"] = remove_outliers(df["price"])
    print(f"Replaced {n_outliers} outlier prices")

    #5) Currency correction for NOK
    df.loc[df["currency"] == "NOK", "price"] *=10.0

    #6) Reports
    hourly = hourly_revenue(df)
    pivot = status_pivot(df)
    ratios = item_refund_ratios(df)


    #7) save output files
    df.to_csv(out_dir / "cleaned_transactions.csv")

    hourly_json = hourly.copy()
    hourly_json.index = hourly_json.index.strftime("%Y-%m-%d %H:%M")
    hourly_json.to_json(out_dir / "hourly_revenue.json", orient = "index", indent =

    pivot.to_csv(out_dir / "status_pivot_table.csv")
    ratios.to_csv(out_dir / "item_refund_ratios.csv")
    print(f"All output files saved to '{out_dir}/'")

    return df, hourly, pivot, ratios
```

In [24]:
```
df, hourly, pivot, ratios = run_pipeline()
```

```
Found 9 log files. Active registers: ['POS_01', 'POS_02', 'POS_03']
data\2026\library_cafe\POS_01\register_sales.log [utf-8] -> 80 records
data\2026\library_cafe\POS_02\register_sales.log [utf-8] -> 80 records
data\2026\library_cafe\POS_03\register_sales.log [utf-8] -> 80 records
data\2026\science_building\POS_01\register_sales.log [utf-8] -> 80 records
data\2026\science_building\POS_02\register_sales.log [utf-8] -> 80 records
data\2026\science_building\POS_03\register_sales.log [utf-8] -> 80 records
data\2026\student_union\POS_01\register_sales.log [utf-8] -> 80 records
data\2026\student_union\POS_02\register_sales.log [utf-8] -> 80 records
data\2026\student_union\POS_03\register_sales.log [utf-8] -> 80 records
Replaced 24 outlier prices
All output files saved to 'output/'
```

In [25]:
```
raw_df = build_dataframe(all_records)
print("Number of 999 outliers:", (raw_df["price"] == 999.0).sum())
```

```
Number of 999 outliers: 24
```

In [26]:
```
df.head(10)
```

Out[26]:

timestamp	register_id	item	price	currency	status
2026-10-15 07:00:21	POS_01	Muffin	32.00	NOK	SUCCESS
2026-10-15 07:00:29	POS_03	Cappuccino	39.30	NOK	SUCCESS
2026-10-15 07:01:05	POS_02	Cappuccino	4.19	USD	SUCCESS
2026-10-15 07:01:18	POS_02	Croissant	41.70	NOK	SUCCESS
2026-10-15 07:01:36	POS_03	Muffin	3.68	EUR	CANCELLED
2026-10-15 07:01:37	POS_03	Muffin	3.65	USD	SUCCESS
2026-10-15 07:01:43	POS_02	Cappuccino	3.85	USD	CANCELLED
2026-10-15 07:01:52	POS_01	Sandwich	7.23	USD	SUCCESS
2026-10-15 07:02:03	POS_03	Croissant	36.00	NOK	REFUND
2026-10-15 07:02:15	POS_03	Espresso	2.79	USD	CANCELLED

In [27]:
```
hourly
```

Out[27]:

timestamp	item	Cappuccino	Croissant	Espresso	Latte	Muffin	Sandwich
2026-10-15 07:00:00		859.75	801.30	694.09	1117.78	981.66	1330.27
2026-10-15 08:00:00		1046.49	790.82	818.15	695.19	743.11	1997.28
2026-10-15 09:00:00		148.91	61.62	49.29	189.14	97.95	233.72

In [28]:
```
pivot
```

Out[28]:

item	status	CANCELLED	REFUND	SUCCESS
Cappuccino		517.86	328.49	1208.80
Croissant		343.81	398.15	911.78
Espresso		336.92	226.66	997.95
Latte		496.55	430.03	1075.53
Muffin		359.48	327.07	1136.17
Sandwich		700.56	726.38	2134.33

In [29]:
```
ratios
```

Out[29]:

item	sales_count	refund_count	refund_ratio
Cappuccino	78	22	0.282051
Croissant	72	23	0.319444
Espresso	79	21	0.265823
Latte	68	24	0.352941
Muffin	75	22	0.293333
Sandwich	80	22	0.275000

Diskussion – Uppgift 5 och resultat

Flödet kombinerar alla funktioner i den ordning som anges i uppgiften. 24 priser på 999.00 ersattes med medianen.

Ordningen på rengöringsstegen: Extremvärden tas bort före NOK-korrigeringen. Om vi först multiplicerade NOK-priserna med 10 skulle de ligga runt 30–80, vilket skulle ändra medelvärdet och standardavvikelsen och göra 3σ-detekteringen mindre tillförlitlig.

Returkvoter: Alla artiklar har en returkvot mellan ungefär 0,27 (Espresso) och 0,35 (Latte). Värdena ligger nära varandra, vilket är väntat: generatorn väljer status slumpmässigt med samma sannolikheter för alla artiklar (4/6 SUCCESS, 1/6 REFUND), vilket ger en förväntad kvot på ungefär 0,25. Skillnaderna mellan artiklarna är alltså slumpmässig variation, inte ett verkligt mönster.

Begränsningar i analysen: Blandade valutor: Intäkten per timme och pivottabellen summerar EUR, NOK och USD tillsammans. Dessa summor är inte meningsfulla som penningbelopp. En bättre lösning vore att räkna om alla priser till en valuta med växelkurser innan summeringen. Global median: Extremvärden ersätts med medianen för *alla* artiklar. En felregistrerad Sandwich (~7,50) får därför ett pris på ungefär 4. Att ersätta med medianen per artikel vore mer korrekt. Returer och avbrutna köp i intäkten:Intäkten per timme inkluderar REFUND- och CANCELLED-transaktioner. En "riktig" intäktsrapport borde troligen bara räkna SUCCESS (och eventuellt dra av returerna).

Användning av generativ AI

Vi använde Gemini som handledare under denna inlämning. Vi använde den för att:

- få en steg-för-steg-plan för uppgiften och rad-för-rad-förklaringar av koden för varje deluppgift (regex, rekursion, NumPy-masker, pandas `resample` / `pivot_table` / `crosstab`),
- felsöka fel, till exempel att installera Python och rätta PATH i Windows, skapa den

virtuella miljön och rätta stavfel och misstag som `np.ndarray` vs `np.asarray` och `match.group.dict()` vs `match.groupdict()`,

- sätta upp Git och GitHub (`.gitignore`, första push, lägga till en medarbetare),
- få ett första utkast till diskussionspunkterna, som vi redigerade och skrev om med egna ord. Utkastet översattes också till svenska.

Vi skrev och körde all kod själva, testade varje funktion separat och kontrollerade att resultaten var rimliga (till exempel 9 filer, 80 poster per fil och de förväntade aktiva kassorna). [Lägg till det ni själva ändrade eller bestämde.]